Project - High Level Design
On
Entertainment Planning Assistant using Agentic AI

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Aman Sharma	EN22CS301109
2	Ananya Bhatia	EN22CS301117
3	Ankit Nagar	EN22CS301134
4	Anusha Singh Panwar	EN22CS301179
5	Arpit Patidar	EN22CS301199

Group Name: Group 02D2

Project Number: AAI-14

Industry Mentor Name: Ashwin Krishnan

University Mentor Name: Prof. Ajeet Singh Rajput

Academic Year: 2026

Table of Contents

1. Introduction
    1.1 Scope of the Document
    1.2 Intended Audience
    1.3 System Overview

2. System Design
    2.1 Application Design
    2.2 Process Flow
    2.3 Information Flow
    2.4 Components Design
    2.5 Key Design Considerations
    2.6 API Catalogue

3. Data Design
    3.1 Data Model
    3.2 Data Access Mechanism
    3.3 Data Retention Policies
    3.4 Data Migration

4. Interfaces
5. State and Session Management

6. Caching

7. Non-Functional Requirements
    7.1 Security Aspects
    7.2 Performance Aspects

8. References

1. Introduction

With the increasing demand for personalized and engaging leisure experiences, planning entertainment activities has become both exciting and time-consuming. Users often struggle to decide what to watch, where to go, or how to organize events based on their preferences, budget, and time constraints.

The Entertainment Planning Assistant using Agentic AI is an intelligent system designed to simplify and automate entertainment planning. By leveraging agentic AI, the system can autonomously understand user preferences, analyze available options, and generate customized entertainment plans such as movie suggestions, outing itineraries, event schedules, or weekend plans.

This system enhances user convenience by providing smart recommendations and complete plans, reducing decision fatigue and improving overall user experience.

1.1 Scope of the Document

This document provides a detailed overview of the Entertainment Planning Assistant using Agentic AI system. It includes:

- Description of system architecture and design
- Explanation of agent-based decision-making and planning
- Functional and non-functional requirements
- Workflow of generating personalized entertainment plans
- Use cases such as trip planning, movie selection, and event scheduling • Limitations and potential future improvements

The document serves as a guide for understanding, developing, and evaluating the system.

Audience 1.2 Intended

This document is intended for:

- Developers and Engineers – to understand system implementation and architecture
- Students and Researchers – to explore applications of agentic AI in planning systems
- Project Evaluators and Faculty – to assess the design and functionality of the system
- End Users and Product Designers – to understand how the system improves entertainment planning

1.3 System Overview

The Entertainment Planning Assistant using Agentic AI is a smart system that helps users plan their entertainment activities efficiently by generating personalized recommendations and structured plans.

The Agentic Entertainment Planner is a Python-based intelligent system designed to orchestrate complex tasks via a multi-step continuous reasoning loop. Upon receiving a high-level goal from a user (e.g., "Generate a weekly schedule of Movie Reviews"), the Planner Agent autonomously decomposes the task, validates its available tools (functions), executes the tools iteratively to gather/generate content, and returns a fully formatted final schedule.

Key Features:

1. Autonomous "ReAct" Reasoning Engine
    - Uses the advanced Reason + Act (ReAct) paradigm. Instead of just following a hard-coded script, the agent continuously *thinks* about what to do next, *executes* a tool, and *observes* the results until the user's high-level goal is completely fulfilled.
2. Semantic In-Memory Vector Database
    - Replaces traditional SQL keyword searching with true "meaning-based" search. Completed task schedules are embedded mathematically using HuggingFace's all-MiniLM-L6-v2 transformer and stored in a highly efficient, CPU-friendly NumPy array using mathematical Cosine Similarity.

Working Overview:

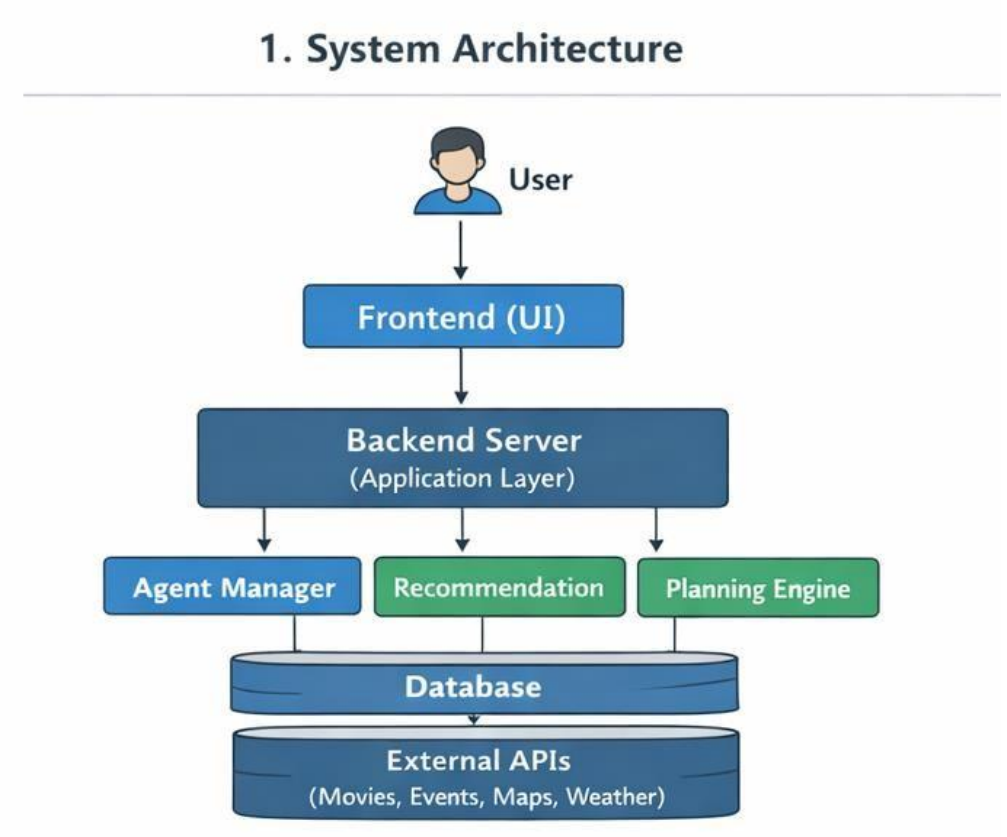
1. User Input: User provides preferences such as interests, budget, time, and location
2. Agent Analysis: AI agents interpret user requirements and available options
3. Plan Generation: The system creates a personalized entertainment plan (e.g., weekend itinerary)
4. Optimization: Plans are refined for better time management and user satisfaction
5. Output Delivery: Final plan is presented with suggestions and alternatives

Applications:

- Weekend or holiday planning
- Movie and show recommendations
- Event and outing scheduling
- Personalized leisure activity planning

2. System Design

The system design of the Entertainment Planning Assistant using Agentic AI focuses on creating a modular, scalable, and intelligent architecture that enables autonomous planning and personalized recommendations. The system integrates user inputs, AI-driven agents, and external data sources to generate optimized entertainment plans.



2.1 Application Design

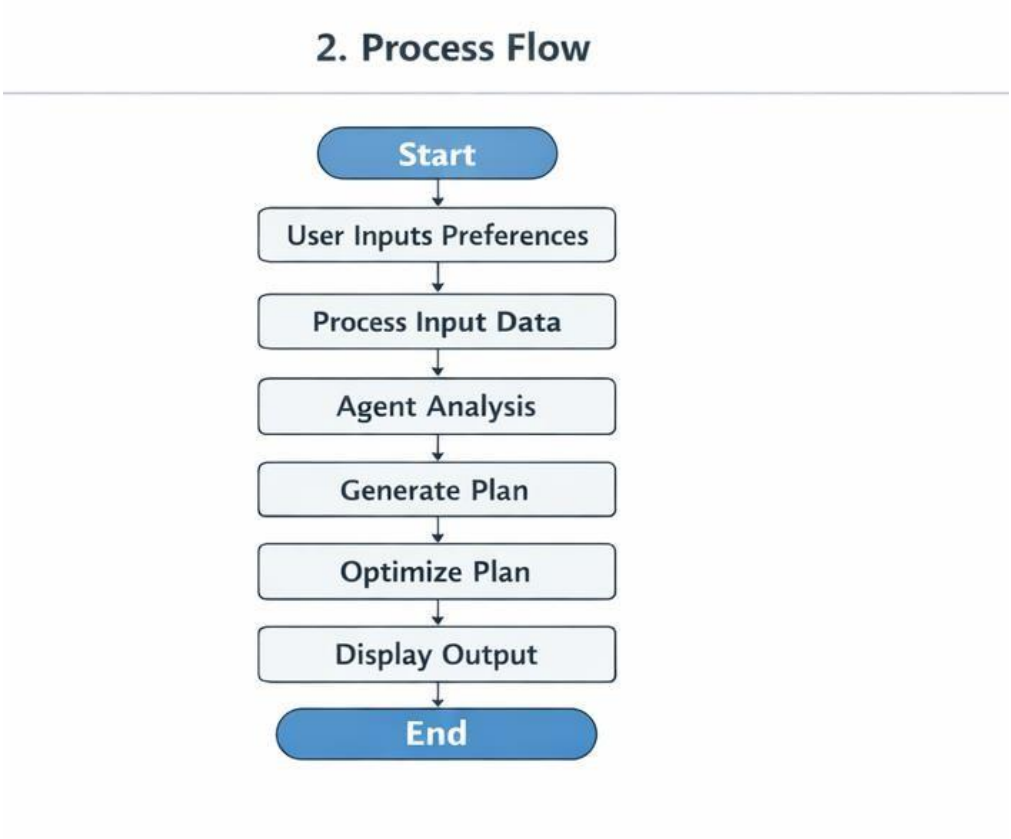
The application follows a highly modular, decoupled architecture consisting of three primary layers:

1. Presentation Layer: A dynamic web interface built with Streamlit, exposing the user inputs and the developer "Execution Logs".
2. Orchestration Layer: A LangChain AgentExecutor wrapping the Google Gemini 2.5 Flash Large Language Model (LLM), enforcing the explicit "Reason and Act" (ReAct) paradigm.
3. Execution Layer: A suite of pure Python functions decorated as LangChain tools, serving as the interface to the mock external world.

2.2 Process Flow

The system follows a step-by-step process:

1. User Input: The target goal is received via the Streamlit UI.
2. Goal Injection: The goal is embedded into an engineered System Prompt and sent to the LLM.
3. Reasoning Loop (ReAct):
    - Thought: The LLM outputs a textual reasoning step (e.g., assessing which tool is required).
    - Action: The LLM specifies a tool name and JSON argument.
    - Execution: The Orchestration layer halts the LLM, executes the local Python tool natively, and captures the result.
    - Observation: The result is injected back into the LLM context.
4. Resolution: The LLM determines the goal is fulfilled, outputs the Final Answer keyword, and breaks the loop.
5. Memory Storage: The final output is vectorized and committed to memory.



2.3 Information Flow

Data moves in a continuous, bidirectional loop between the User Interface (Streamlit), the Central Orchestrator (LangChain), and the Language Model (Google Gemini 2.5 Flash).

The flow follows this continuous cycle:

1. User Request (UI to Orchestrator): The user inputs a high-level goal in natural language via the front-end dashboard.
2. Context Delivery (Orchestrator to LLM): The Orchestrator packages the user's goal along with a list of available tool schemas (JSON definitions of what the tools can do) and sends it to the Gemini LLM.
3. Action Decision (LLM to Orchestrator): The LLM analyzes the context and returns a specific command indicating which tool to execute and what inputs to provide it.
4. Tool Execution (Orchestrator to Local System): The Orchestrator pauses the LLM, reads the command, and executes the designated Python tool locally.
5. Observation Feedback (Local System to LLM): The data generated by the tool is captured by the Orchestrator and fed back into the LLM as an "Observation".

(Steps 3, 4, and 5 loop continuously until the LLM resolves the overarching goal and returns the final textual schedule back to the User).

2.4 Components Design

The system is divided into the following components:

- app.py: The entry point. Handles environment loading (.env), manages the Streamlit UI columns, and invokes the agent.
- planner_agent.py: Contains the PlannerAgent class. Instantiates ChatGoogleGenerativeAI, binds the tool schemas, and manages the LangChain AgentExecutor.
- mock_tools.py: Houses the isolated Python functions representing external services fetch_trending_movies, get_movie_metadata, write_movie_review, generate_publishing_schedule).
- vector_memory.py: A Custom built in-memory vector database for semantic logging using NumPy and HuggingFace models.

2.5 Key Design Considerations

While designing the system, the following factors are considered:

- Scalability: Ability to handle multiple users simultaneously
- Personalization: Accurate understanding of user preferences
- Modularity: Easy to update or replace components
- Performance: Fast response time for real-time recommendations
- Usability: Simple and intuitive user interface
- Zero dependency Vector DB : Raw NumPy arrays were chosen over FAISS or Pinecone to ensure 100% cross-platform compatibility without heavy C++ or cloud dependencies

2.6 API Catalogue

While the system uses mock interface tools, their design mathematically mimics REST APIs, ensuring they are easily upgradeable.

- Tool Interface 1: fetch_trending_movies() -> list[str]
- Tool Interface 2: get_movie_metadata(movie_name: str) -> dict
- Tool Interface 3: write_movie_review(movie_name: str, metadata: str) -> str
- System API: Google Gemini API (gemini-2.5-flash) (External via RPC/REST for the LLM Brain).

3. Data Design

The data design of the Entertainment Planning Assistant using Agentic AI focuses on efficiently storing, retrieving, and managing user and entertainment-related data. It ensures data consistency, scalability, and personalization.

3.1 Data Model

The system follows a relational + semi-structured data model:

Key Entities:

- User: UserID, Name, Preferences, Location, Budget
- Entertainment Item: ItemID, Type (Movie/Event/Place), Name, Genre, Rating
- Plan: PlanID, UserID, Activities, Schedule, Cost

Relationships:

- A User can have multiple Plans
- A Plan contains multiple Entertainment Items
- A User provides Feedback on plans

3.2 Data Access Mechanism

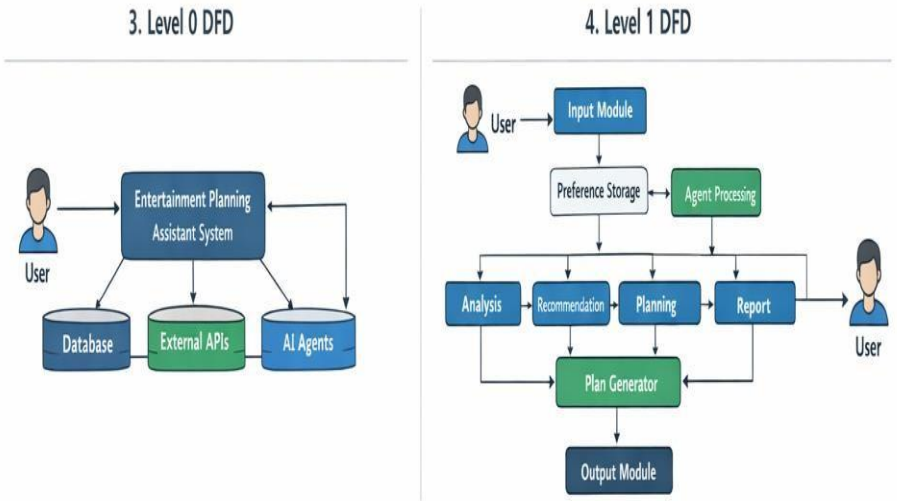
- Embedding Generation: SentenceTransformers (all-MiniLM-L6-v2) processes text input into embeddings locally on the CPU.
- Similarity Search: Incoming search queries are embedded, and a mathematical Cosine Similarity (np.dot) operation dictates retrieval against the vectors array, returning the index of the highest-scoring raw text.

3.3 Data Retention Policies

As an academic prototype, the system operates with volatile persistence. All states, logs, and vectorized memories are stored solely in RAM (st.session_state and local list classes) and are wiped cleanly upon application termination or hard refresh.

3.4 Data Migration

N/A. As the system relies on transient RAM architecture and mock databases, no historical data migration schemas are required.



4. Interfaces

- Human-Computer Interface (HCI): A Streamlit Split-Pane Web Dashboard. The left pane provides user inputs; the right pane displays developer execution logs.

- System-to-System Interface: LangChain communicates with Google generative backend services via gRPC / HTTP REST mapped through the langchain-google-genai package.

5. State and Session Management

State is managed exclusively by the Streamlit session_state singleton dictionary.

- st.session_state.memory: Persists the VectorMemory object instance across button interactions.

- st.session_state.run_log: Captures and retains the intermediate reasoning steps of the LangChain Executor to ensure the UI visually displays the agent's thought process without refreshing entirely.

6. Caching

Caching improves performance and reduces latency:

- In-memory caching (e.g., Redis) for quick data access
- Cache frequently accessed data:
  - Popular movies/events
  - User preferences
- Cache invalidation strategies:
  - Time-based expiration
  - Update on data change

7. Non-Functional Requirements

7.1Security Aspects

- Masking: The Google Gemini API key is isolated entirely from the source code and the front-end user interface. It is accessed natively via OS environment variables leveraging standard .env parsing (python-dotenv), which is strictly excluded (.gitignore) from version control.

- Execution Safety: The mock tools execute in isolated Python scopes, mitigating standard injection risks.

7.2. Performance Aspects

- Throughput: The system utilizes Google's gemini-2.5-flash model, specifically selected because of its low-latency, high-Token-Per-Second (TPS) output necessary for rapid iteration within ReAct loops.

- Local Scalability: By designing the memory module with direct NumPy mathematical operations, the Cosine Similarity search operates with O(n) complexity natively, resulting in microsecond retrieval times.

8. References

Google Python SDK Documentation (google-generativeai, gemini-2.5-flash)
LangChain Agents Architecture Reference (create_react_agent)
HuggingFace Sentence Transformers Documentation (all-MiniLM-L6-v2)
Streamlit Session State Constraints Formulation.